

APGO – Architecture du monorepo

Version du 2 octobre 2026 · Décisions validées : monorepo Nx, application admin séparée, composants Angular sans suffixe .component, Bible au format USFM

Un seul dépôt Nx réunit l'API NestJS, le site Angular, l'administration Angular et l'application Flutter. L'API est décrite une seule fois (OpenAPI) et génère les clients TypeScript et Dart ; les couleurs et polices sont définies une seule fois et partagées entre web et mobile.

1. Structure des dossiers

```
apgo/
├── apps/
│   ├── api/           NestJS : l'API unique
│   ├── web/           Angular SSR : site public, espace apprenant et formateur
│   ├── admin/         Angular : administration et service client
│   └── mobile/         Flutter
├── libs/              Code TypeScript partagé
│   ├── api-client/    Client TypeScript généré depuis l'OpenAPI
│   ├── ui/            Composants Angular communs
│   ├── i18n/          Traductions Transloco (fr, puis gouro)
│   └── util/           Fonctions utilitaires
├── packages/          Code Dart partagé
│   ├── apgo_api/      Client Dart généré depuis l'OpenAPI
│   └── apgo_design/    Couleurs et polices générées pour Flutter
├── design/
│   ├── tokens/        Source unique des couleurs, polices, espacements (JSON)
│   └── fonts/          Poppins, Inter, Source Serif 4
├── content/
│   └── bible/          Texte source USFM et index des fichiers audio
├── tools/
│   ├── bible-import/  USFM → base, recherche, paquets hors-ligne
│   ├── media-migration/ Import du 1 To vers R2, avec reprise
│   └── codegen/        Génération des clients API et des tokens
├── infra/
│   ├── docker-compose.yml PostgreSQL, Redis, Meilisearch en local
│   └── deploy/         Coolify, sauvegardes
├── docs/              Cahier des charges, courriers, architecture
├── .claude/           Commandes et skills pour Claude Code
├── .github/workflows/ CI : tests, lint, build, déploiement
├── CLAUDE.md          Conventions du projet
├── nx.json
└── package.json
```

2. L'API (apps/api)

Un module NestJS par domaine du cahier des charges, et une couche d'infrastructure derrière des interfaces pour pouvoir changer de prestataire sans toucher au reste.

```

src/modules/
  auth/          Connexion, jetons, double authentification
  users/         Comptes et profils
  access/        Rôles et permissions configurables
  media/         Médiathèque
  courses/       Formations, modules, leçons, progression
  exams/         QCM, tentatives, notes
  certificates/  PDF, numéro unique, QR code, vérification
  bible/         Livres, chapitres, versets, audio, paquets hors-ligne
  payments/      Interface unique, faux prestataire, interrupteur
  support/       Tickets, FAQ, contact
  audit/         Journal des actions des admins
  notifications/ Emails, Firebase
src/infrastructure/
  storage/       MediaStorage : R2, Cloudinary
  video/         VideoProvider : YouTube, Jesus Film Project, plus tard Bunny Stream
  search/        Meilisearch
  queue/         BullMQ
  prisma/        Schéma et migrations

```

3. Le web et l'admin (apps/web, apps/admin)

Deux applications Angular qui partagent `libs/ui`, `libs/i18n` et `libs/api-client`. Le site public reste léger et bien référencé ; l'administration n'est jamais servie au public. L'espace formateur reste dans `web`.

Convention de nommage

Type	Fichiers	Classe
Composant	<code>gadiel-list.html</code> , <code>.ts</code> , <code>.scss</code> , <code>.spec.ts</code>	<code>Gadiellist</code>
Service	<code>gadiel.service.ts</code>	<code>GadielService</code>
Modèle	<code>gadiel.model.ts</code>	—
Routes	<code>gadiel.routes.ts</code>	—
Guard, pipe, directive, intercepteur	<code>auth.guard.ts</code> , <code>date.pipe.ts</code> ...	<code>AuthGuard...</code>

Organisation par entité

```

gadiel/
├── add/    gadiel-add.html · gadiel-add.ts · gadiel-add.scss · gadiel-add.spec.ts
├── list/   gadiel-list.html · gadiel-list.ts · gadiel-list.scss · gadiel-list.spec.ts
├── show/   gadiel-show.html · gadiel-show.ts · gadiel-show.scss · gadiel-show.spec.ts
├── gadiel.service.ts
├── gadiel.model.ts
└── gadiel.routes.ts

```

4. Le mobile (apps/mobile)

Organisation par fonctionnalité (`lib/features/media`, `courses`, `bible`, `account`...), avec les packages Dart de packages/ en dépendances locales. Nx ne gère pas Flutter nativement : on lui ajoute des commandes (`nx run mobile:build`, `nx run mobile:test`) pour garder une seule CI.

5. La Bible

La Bible n'est pas une application à part : c'est un contenu qui traverse le projet, du fichier source jusqu'aux écrans.

1. **Source** (`content/bible/`) : le texte gourou au format USFM, un fichier par livre. Les fichiers audio, un par chapitre, sont stockés sur R2 ; le dépôt ne garde que leur index.
2. **Import** (`tools/bible-import/`) : un script remplit PostgreSQL (livres, chapitres, versets), indexe les versets dans Meilisearch et génère un paquet hors-ligne par livre (SQLite compressé, taille connue), envoyé sur R2.
3. **API** (module `bible`) : texte, liens audio, liste des paquets avec leur taille, dernière position de lecture.
4. **Clients** : lecture en ligne référencée sur Google côté web (une URL par chapitre) ; téléchargement livre par livre et lecture hors-ligne côté mobile.

Mettre à jour une traduction revient à relancer le script, sans redéployer les applications. Plusieurs versions peuvent coexister (édition de 1979, nouvelle traduction en cours).

Provenance du texte

Option	Décision
Fichiers officiels USFM/USX de l'Alliance Biblique de Côte d'Ivoire	Retenue — demande envoyée via l'équipe de traduction
Conversion d'un PDF ou Word	Secours uniquement
Copie depuis YouVersion	Exclue (droits d'auteur)

En attendant les fichiers, le module est développé avec la Louis Segond 1910 (domaine public, USFM).

6. Sources partagées

- **OpenAPI** : l'API génère son schéma, qui génère `libs/api-client` (TypeScript) et `packages/apgo_api` (Dart).
- **Design tokens** : `design/tokens` génère les variables CSS pour Angular et un fichier Dart pour Flutter (violet #5B2A86, blanc, jaune doré #D4A017, violet foncé #3E1B5E, violet très clair #F4EEFA ; Poppins, Inter, Source Serif 4).

7. Ressources externes demandées

Partenaire	Ressources	Intégration
Alliance Biblique de Côte d'Ivoire	Texte USFM, audio par chapitre, autorisation	<code>tools/bible-import</code> , module <code>bible</code>
Jesus Film Project	Accès API, films et clips en gourou et en français	Connecteur <code>VideoProvider</code>